Research Plan - Graduation Project AI&ES

Title graduation project: Network Latency in Distributed LLM Inference
Student name: Kelsey Penners
Student ID number: 2198525
Department: Electrical Engineering
Research group: Signal Processing Systems
Thesis supervisor: Alex Alvarado
Date: February 20, 2026

1 Background

The training and inference processes of frontier large language models (LLMs) now require unprecedented resources, with the training of OpenAI's GPT-4 model using 25,000 NVIDIA A100 GPUs and inference using multiple clusters of 128 GPUs distributed across different datacenters [13]. Although the training of these models is resource intensive with high upfront costs, the inference phase is continuous, with successful AI applications generating costs often exceeding training due to cumulative predictions. Distributed deployment has become a requirement for serving large-scale LLMs.

Most modern LLMs are transformer-based models equipped with attention mechanisms that ensure the model has the language context to accurately determine input meaning and predict output [11]. To distribute models across GPUs and machines, parallelization strategies such as data parallelism, tensor parallelism, and pipeline parallelism are used, distributing training or inference workloads in various dimensions across the model and data. These parallelization methods allow for the scaling we see in contemporary models, but require frequent communication between GPUs to exchange parameters, activations, and intermediate calculations. Collective communication algorithms such as AllReduce and point-to-point transfers move data across GPUs and machines carried by hardware interconnects, including NVLink, PCIe, and Ethernet-based networking. The characteristics of the network impact the performance of an LLM by introducing communication delays and unexpected bottlenecks [8].

Existing research understandably focuses on optimizing distributed *training*, where communication costs are inflated due to large batches and long training runs. Optimization of these models is widely explored, all in efforts to hide the inherent communication latency present in distributed systems. However, distributed *inference* has received significantly less attention, despite it directly affecting user experience, cost of operation, and system design. Training and inference are model processes with distinct compute and communication patterns, so they behave differently in response to network latency. Specific inference processes, GPU compute capability, and interconnect technology complicate performance characteristics even further. Investigating how network latency and interconnect quality influence distributed LLM inference can better inform system design strategies for large-scale AI systems.

2 Literature Survey

2.1 LLM Architecture and Inference Phases:

Large language models are transformer-based models that use attention mechanisms in order to make accurate predictions with natural language input [11]. During inference, input text is broken down into fragments named tokens and mapped to vector embeddings. Each token gets projected into query (Q), key (K), and value (V) vectors used to compute attention weights. In the **prefill** phase, all K and V vectors for input tokens are computed and stored in the key-value (KV) cache. Following prefill is the **decode** phase, consisting of autoregressive generation of output tokens, where attention is computed for each new token and appended to the KV cache. Prior work has shown that prefill is generally compute bound, through the processing and computations of Q, K and V vectors, while decode is often memory-bound due to its autoregressive nature,

requiring the loading of all model weights and the full KV cache at every step [5]. The inherent differences between the two phases result in distinct behavior when deployed in distributed systems.

2.2 Parallelization Strategies & Communication Patterns:

Parallelization strategies, such as data, tensor, and pipeline parallelism, allow LLMs to run across multiple GPUs and multiple nodes by distributing computations across available hardware [2].

Data parallelism replicates model weights and splits data across available GPUs. While a crucial step in distributed training, it is less relevant in distributed inference due to the lower batch sizes. [3].

Tensor parallelism divides parameter tensors of each layer across multiple GPUs, such that each GPU has a partition of the model weights [10]. Using AllReduce operations allows for synchronization of intermediate activations across GPUs. As AllReduce communication has been shown to consume a significant portion of iteration time in distributed workloads, tensor parallelism is usually restricted to GPUs within a single node, where high-bandwidth interconnnects such as NVLink can prevent significant performance degradation. [1,8].

Pipeline parallelism distributes a model by allocating groups of layers across devices with activations forwarded between stages using point-to-point transfers [4]. Unlike tensor parallelism, which requires frequent communication between GPUs, pipeline parallelism only communicates between stages, making it a food alternative to tensor parallelism when dealing with lower-bandwidth interconnects such as Ethernet [1].

Each parallelization method has their own communication patterns with different sensitivity to network latency, resulting varied impact on performance depending on how LLMs are distributed.

2.3 Distributed Inference Systems

Distributed inference introduces memory and serving challenges that are not seen in training, resulting in the development of inference optimization frameworks. One such framework, vLLM, introduced PagedAttention, which manages the KV cache using virtual memory paging, eliminating memory fragmentation and improving inference throughput [6]. DeepSpeed Inference also demonstrated that distributed inference requires strategic parallelism schemes using 3D or hybrid parallelism, combining both tensor and pipeline parallelism to serve models at large-scale [1]. Frameworks like these optimize inference throughput and latency and treat communication overhead as a cost to minimize, not a variable to isolate.

2.4 Communication Overhead & Network Latency in Distributed LLMs

Recent work has been done characterizing communication behavior in distributed LLM inference. Xu et al. found that commmunication overhead takes up a larger portion of total execution time in inference than in training, and that the decode phase produces significantly higher communication frequency than prefill [12]. There is limited compute-communication overlap during autoregressive generations, unlike prefill where computation partially masks communication. These findings suggest that inference – specifically decode – is more latency-sensitive than training.

Further work on communication in distributed LLMs is majorly training focused. Qi et al. demonstrate that interconnect type has an impact on performance, showing RDMA outperforms TCP/IP Ethernet by up to 4.79x, although this analysis was found examining training workloads [8]. Similarly, Mi et al. explored the effect of Ethernet link latency on training time and found minimal impact on training performance [7], leaving open whether the same holds for inference.

Existing work characterizes communication patterns in fixed hardware setups, but has not been studied under degraded network conditions. Limited work has isolated network latency as the independent variable across inference phases and parallelism strategies. This project addresses this gap by measuring inference performance under controlled latency conditions, isolating network latency as the independent variable.

3 Research Questions

Building on the motivations expressed, this project aims to address the following main research question:

How does network communication latency affect distributed LLM inference performance across different hardware interconnects and cluster configurations?

With subquestions:

- How does increasing communication latency affect prefill and decode phases differently?
- At what point does measurable degradation occur in performance metrics such as time to first token (TTFT) and inter-token latency (ITL)?
- How does inference performance degrade when scaling across heterogeneous clusters with mixed interconnects, compared to homogeneous intra-node setups?

Hypotheses: The decode phase will be significantly more sensitive to network latency than the prefill phase due to higher communication frequency during autoregressive generation, as characterized in prior work [12]. Additionally, inference workloads are expected to show different latency sensitivity compared to training workloads, particularly when scaling across lower-bandwidth Ethernet interconnects.

4 AI&ES Relevance

This project sits perfectly at the intersection of Artificial Intelligence and Engineering Systems by analyzing how large language models behave when deployed on distributed hardware infrastructure. By examining specific LLM characteristics – such as transformer architecture, compute patterns of prefill and decode phases, and parallelization strategies – this thesis explores how these models interact with physical system constraints like network latency, hardware topology, and GPU interconnects. Through hardware-aware performance evaluation of distributed models, this research provides insight into the quintessential engineering challenge of turning AI algorithms into scalable, real-world systems. Ultimately, this project will show that AI deployment boils down to a systems engineering problem, where model requirements and network characteristics must be co-optimized for utmost efficiency.

5 Research Methods

This thesis will be completed by implementing open-source models on GPU servers using the vLLM framework to distribute and perform inference evaluations [6]. The work shall be split into 3 phases: single GPU, intra-node, and inter-node distributed inference. Across all phases, key performance metrics to be analyzed include time-to-token (TTFT), inter-token latency (ITL), and throughput, allowing prefill and decode phase sensitivity to be observed independently [9].

The two main servers to be used are funnotch and shy-fec, belonging to the ICT group at TU/e, consisting of 4xV100-32GB connected via PCIe and 8xA100-40GB connected via NVLinks, respectively. An array of Meta's open-source LLaMa models will be deployed, with varying parameter counts, architectures, memory requirements, and inference constraints.

Phase 1 consists of single GPU performance baselines using smaller LLaMa models such as Llama-3.2-1B and Llama-3.1-8B, benchmarked on both a V100 and A100 GPU via vLLM for comparison across GPU generations. Input and output sequence length impact on performance will be explored.

Phase 2 scales up to multi-GPU experiments, allowing for evaluation of larger models such as Llama-3.3-70B Instruct and Llama 4 Scout. Both servers will be evaluated to investigate impact of PCIe versus NVLink interconnects on inference performance. Latency injection will be implemented through software to introduce controlled delays in communication primitives, isolating the effect of intra-node communication latency to observe impact on LLM inference performance.

Phase 3 explores heterogeneous inter-node distribution, deploying models across Ethernet links. Inter-node network latency injection will use Linux traffic control with network emulation to simulate varying Ethernet latencies, characterizing how inference performance degrades and how mixed interconnect topologies affect the prefill and decode phases.

To ensure quality and reliability of data, experimental configurations will be run multiple times and averaged, and thorough documentation of performed experiments will be maintained.

Performance will be analyzed through visualization of communication latency and throughput relationships,

comparison of metrics corresponding to specific phases, and identification of performance degradation thresholds.

6 Expected Outcomes

This project expects to provide a characterization of how network latency impacts distributed LLM performance across different interconnect types and cluster configurations. Potential findings include identifying latency thresholds where performance meaningfully degrades, and whether the decode phase is significantly more sensitive to communication latency than prefill, suggested by existing communication charcaterization work [12]. From experiments, we'll produce an inference performance degradation curve as a function of network latencies across configurations.

Results may also reveal practical guidelines for distributed inference deployment, such as when scaling across Ethernet links becomes a performance bottleneck. Broadly, this work may reveal that inference and training have different network sensitivities, with implications for data center design decisions and resource allocation for LLM serving.

7 Time Planning

The 27 week project phase (February 23 - August 28) is outlined in Figure 1. The first six weeks, not included in the Gantt chart, were used to perform a comprehensive literature search, environments set up, and initial tool exploration.

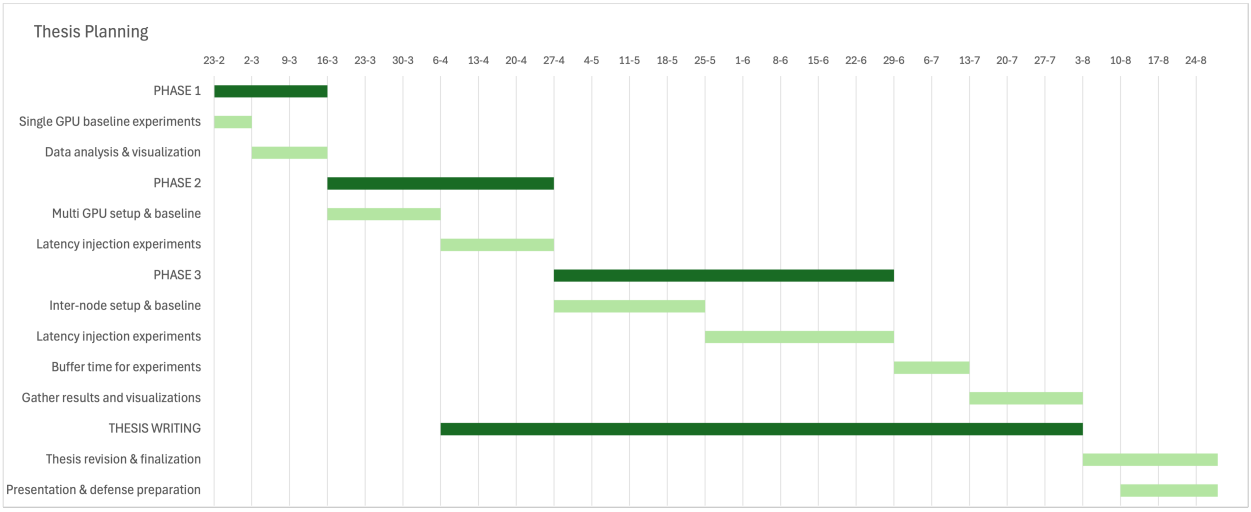


Figure 1: Overview of project timeline

Phase 1 (weeks 7-9) to obtain single GPU baselines, phase 2 (weeks 10-15) for multi-GPU tensor parallelism with latency injection exeriments, and phase 3 (weeks 16-24) for inter-note network latency experiments using pipeline parallelism. There are two weeks of buffer time in case needed to handle server availability, and other possible delays. Thesis writing will be done parallel to phases, then will be revision (weeks 30-33) and defense preparation (weeks 31-33).

References

[1] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. Deepspeed inference: Enabling efficient inference of transformer models at unprecedented scale, 2022.

[2] Jiangfei Duan, Shuo Zhang, Zerui Wang, Lijuan Jiang, Wenwen Qu, Qinghao Hu, Guoteng Wang, Qizhen Weng, Hang Yan, Xingcheng Zhang, Xipeng Qiu, Dahua Lin, Yonggang Wen, Xin Jin, Tianwei Zhang, and Peng Sun. Efficient training of large language models on distributed infrastructures: A survey, 2024.

[3] Jared Fernandez, Luca Wehrstedt, Leonid Shamis, Mostafa Elhoushi, Kalyan Saladi, Yonatan Bisk, Emma Strubell, and Jacob Kahn. Hardware scaling trends and diminishing returns in large-scale distributed training, 2025.

[4] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism, 2019.

[5] Joyjit Kundu, Wenzhe Guo, Ali BanaGozar, Udari De Alwis, Sourav Sengupta, Puneet Gupta, and Arindam Mallik. Performance modeling and workload analysis of distributed large language model training and inference. In *2024 IEEE International Symposium on Workload Characterization (IISWC)*, pages 57–67, 2024.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023.

[7] Guangcan Mi, Xiang He, and Xiaolong Zheng. Relevance of latency in ethernet networking for ai infrastructure. In *ECOC 2024; 50th European Conference on Optical Communication*, pages 1299–1302, 2024.

[8] Hao Qi, Liuyao Dai, Weicong Chen, Zhen Jia, and Xiaoyi Lu. Performance characterization of large language models on high-speed interconnects. In *2023 IEEE Symposium on High-Performance Interconnects (HOTI)*, pages 53–60, 2023.

[9] Shweta Salaria, Zhuoran Liu, and Nelson Mimura Gonzalez. Meta-metrics and best practices for system-level inference performance benchmarking, 2025.

[10] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[12] Lang Xu, Kaushik Kandadi Suresh, Quentin Anthony, Nawras Alnaasan, and Dhabaleswar K. Panda. Characterizing communication patterns in distributed large language model inference, 2025.

[13] Minxian Xu, Junhan Liao, Jingfeng Wu, Yiyuan He, Kejiang Ye, and Chengzhong Xu. Cloud native system for llm inference serving, 2025.